TD4 : Comparaison expérimentale d'algorithmes de tri

Rafael Rodrigues Alexis Lacombe

DL3 Gestion - Informatique
5 novembre 2025

Repository GitHub :
github.com/rdsrafa/ProjetAlgo

Table des matières

1 Introduction **3**

2 Environnement de test **3**
2.1 Configuration matérielle et logicielle . . . 3
2.2 Protocole expérimental . . . 3

3 Rappels théoriques **4**
3.1 Complexités temporelles . . . 4

4 Résultats expérimentaux **4**
4.1 Tableaux aléatoires . . . 4
4.1.1 Focus sur les algorithmes performants . . . 5
4.2 Tableau trié croissant (meilleur cas) . . . 5
4.3 Tableau trié décroissant (pire cas) . . . 6
4.3.1 Graphique global . . . 6
4.3.2 Graphiques séparés . . . 7
4.4 Tableau constant . . . 7
4.5 Comparaison des trois versions du tri à bulle . . . 8

5 Analyses spécifiques par algorithme **9**
5.1 Analyse spécifique : Tri à bulle v1 . . . 9
5.2 Analyse spécifique : Tri à bulle v2 . . . 10
5.3 Analyse spécifique : Tri à bulle v3 . . . 11
5.4 Analyse spécifique : Tri cocktail . . . 12
5.5 Analyse spécifique : Tri par sélection . . . 13
5.6 Analyse spécifique : Tri par insertion . . . 14
5.7 Analyse spécifique : Tri par comptage . . . 15
5.8 Analyse spécifique : Tri fusion . . . 16
5.9 Analyse spécifique : Tri rapide . . . 17
5.10 Cas particulier : Tri par comptage . . . 18

6 Analyse comparative et synthèse **19**
6.1 Confrontation théorie vs pratique . . . 19
6.2 Comportements surprenants . . . 19
6.3 Algorithme le plus performant en pratique . . . 20

7 Conclusion **20**

1 Introduction

Ce rapport présente une étude expérimentale comparative de différents algorithmes de tri. L'objectif est d'évaluer leurs performances en fonction de la taille des données et du type de configuration (tableaux triés, inversés, constants, aléatoires), et de confronter les résultats obtenus aux complexités théoriques étudiées en cours.

Les algorithmes implémentés et testés sont :
- Tri à bulle (3 versions : basique, optimisation de boucle, optimisation d'arrêt)
- Tri cocktail
- Tri par sélection
- Tri par insertion
- Tri par comptage (dénombrement)
- Tri fusion
- Tri rapide

2 Environnement de test

2.1 Configuration matérielle et logicielle

Les tests ont été réalisés sur deux machines Lenovo avec des configurations similaires :

Machines	Lenovo (2 configurations)
Processeurs	AMD Ryzen 7 / AMD Ryzen 5
Mémoire RAM	16 GB
Système d'exploitation	Windows 11 avec WSL (Windows Subsystem for Linux) - Ubuntu
Langage	C
Compilateur	GCC (via WSL Ubuntu)
Options de compilation	`gcc -O2 -Wall`
IDE	Visual Studio Code
Outil de mesure	`clock()` de `<time.h>`
Outil graphique	gnuplot 5.4

Table 1 – Environnement de test

2.2 Protocole expérimental

Les tests ont été effectués sur des tableaux de tailles variables : **1000, 2000, 3000, 4000, 5000, 10000, 15000, 20000, 25000** éléments.

Pour chaque taille, quatre scénarios ont été testés :

1. **Tableau trié croissant** : $[0, 1, 2, ..., n-1]$
2. **Tableau trié décroissant** : $[n-1, n-2, ..., 1, 0]$
3. **Tableau constant** : tous les éléments identiques
4. **Tableaux aléatoires** : moyenne sur 500 tableaux générés aléatoirement

Deux plages de valeurs ont été considérées :
- **Petites valeurs** : $[0, 100]$
- **Grandes valeurs** : $[0, 1\,000\,000]$

3 Rappels théoriques

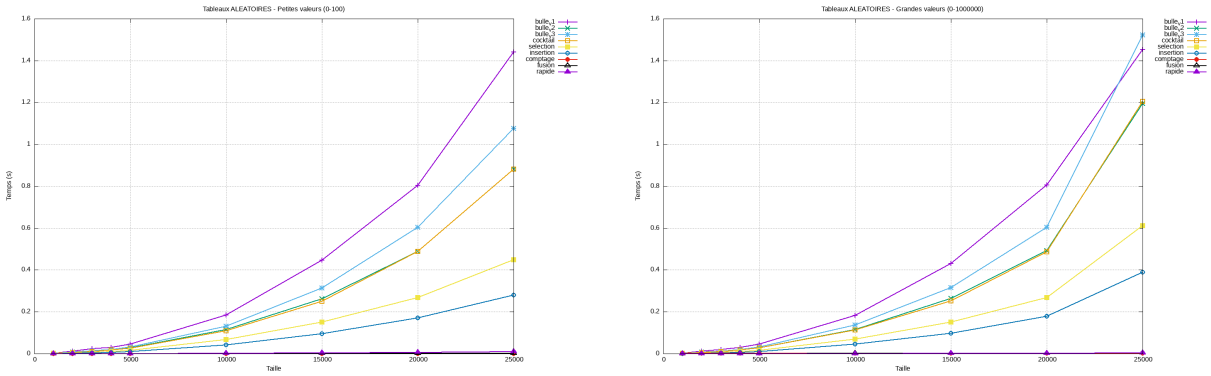
3.1 Complexités temporelles

Algorithme	Meilleur cas	Cas moyen	Pire cas	En place	Stable
Tri à bulle (v1)	$O(n^2)$	$O(n^2)$	$O(n^2)$	Oui	Oui
Tri à bulle (v2)	$O(n^2)$	$O(n^2)$	$O(n^2)$	Oui	Oui
Tri à bulle (v3)	$O(n)$	$O(n^2)$	$O(n^2)$	Oui	Oui
Tri cocktail	$O(n)$	$O(n^2)$	$O(n^2)$	Oui	Oui
Tri par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$	Oui	Non
Tri par insertion	$O(n)$	$O(n^2)$	$O(n^2)$	Oui	Oui
Tri par comptage	$O(n+k)$	$O(n+k)$	$O(n+k)$	Non	Non
Tri fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Non	Oui
Tri rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Oui	Non

TABLE 2 – Complexités théoriques et propriétés (où k = plage de valeurs)

4 Résultats expérimentaux

4.1 Tableaux aléatoires



(a) Petites valeurs (0-100) (b) Grandes valeurs (0-1M)

FIGURE 1 – Performances sur tableaux aléatoires

Observations :

— Le tri à bulle v1 (courbe violette) est le plus lent, atteignant environ 1.45s à 25000 éléments. Les versions v2 et v3 suivent avec des temps légèrement inférieurs (environ 0.9s et 1.1s respectivement).

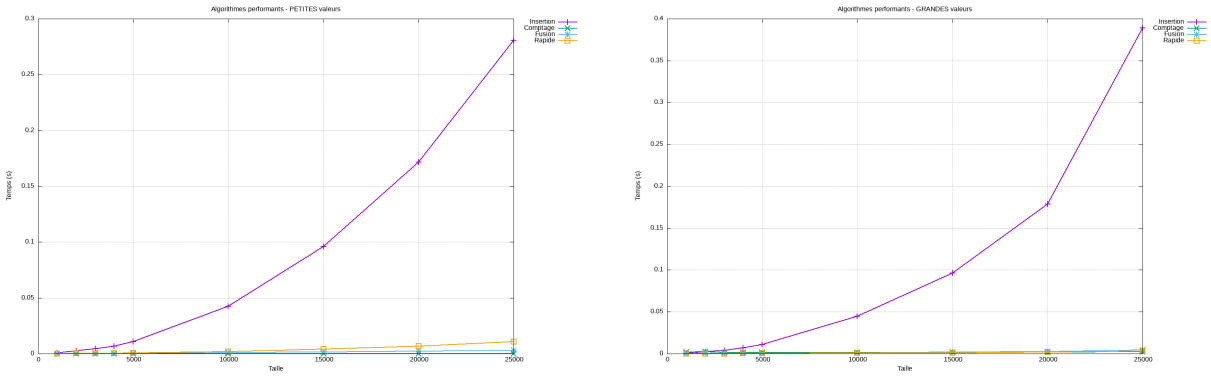
— Le tri cocktail (jaune) et le tri par sélection (jaune clair) montrent des performances intermédiaires autour de 0.45s à 25000 éléments, significativement meilleures que les tris à bulle.

— Le tri par insertion (bleu clair) présente une courbe parabolique atteignant environ 0.3s à 25000 éléments, le plaçant parmi les algorithmes quadratiques les plus efficaces.

— Les algorithmes les plus performants sont le tri fusion, le tri rapide et le tri par comptage : leurs courbes restent pratiquement plates à l'échelle du graphique, avec des temps proches de 0s même pour 25000 éléments.

— Entre petites et grandes valeurs, la principale différence visible concerne le tri par comptage : avec les grandes valeurs, sa courbe devient légèrement visible (temps non nul) alors qu'elle était invisible avec les petites valeurs.

4.1.1 Focus sur les algorithmes performants



(a) Petites valeurs (b) Grandes valeurs

FIGURE 2 – Comparaison des algorithmes performants

Analyse :

— Le tri par insertion (courbe violette) montre clairement sa complexité $O(n^2)$ avec une croissance parabolique, atteignant environ 0.28s à 25000 éléments pour les petites valeurs.

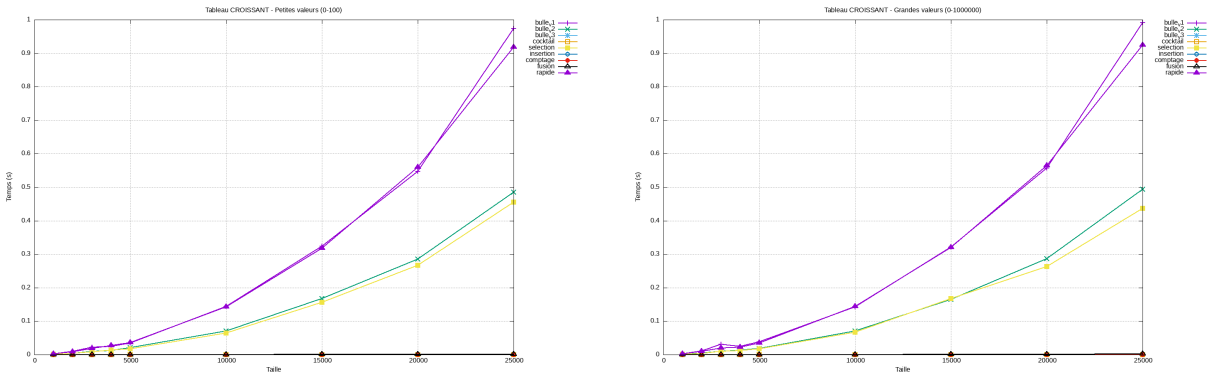
— Le tri rapide (courbe jaune/orange) affiche une croissance très légère, passant de 0s à environ 0.008s à 25000 éléments, confirmant sa complexité $O(n \log n)$.

— Le tri fusion (courbe cyan) et le tri par comptage (courbe verte) sont quasiment confondus et restent proches de 0s sur toute la plage testée.

— Avec les grandes valeurs, le tri par comptage reste performant mais légèrement moins que le tri fusion, alors qu'avec les petites valeurs ils étaient équivalents.

— Le tri par insertion reste le moins performant de ce groupe, mais son écart avec les algorithmes $O(n \log n)$ est clairement visible sur cette échelle réduite.

4.2 Tableau trié croissant (meilleur cas)



(a) Petites valeurs (0-100) (b) Grandes valeurs (0-1M)

FIGURE 3 – Performances sur tableau croissant (meilleur cas)

Observations :

- Le tri à bulle v1 (violette) reste le plus lent avec environ 0.97s à 25000 éléments, démontrant qu'il ne bénéficie pas du tout de l'ordre initial du tableau.
- Le tri rapide (violet foncé) montre une dégradation importante, atteignant environ 0.92s à 25000 éléments, proche du tri à bulle v1. Cela révèle son pire cas sur données triées avec un pivot mal choisi.
- Le tri à bulle v2 (vert) et le tri cocktail (jaune) affichent des performances intermédiaires autour de 0.45-0.5s.
- Le tri à bulle v3, le tri par insertion, le tri par comptage et le tri fusion ont des courbes plates proches de 0s. Pour bulle v3 et insertion, cela confirme leur capacité à détecter un tableau déjà trié en $O(n)$.
- L'écart entre petites et grandes valeurs est négligeable pour tous les algorithmes, ce qui confirme que les algorithmes par comparaison sont insensibles à la magnitude des valeurs.

4.3 Tableau trié décroissant (pire cas)

4.3.1 Graphique global

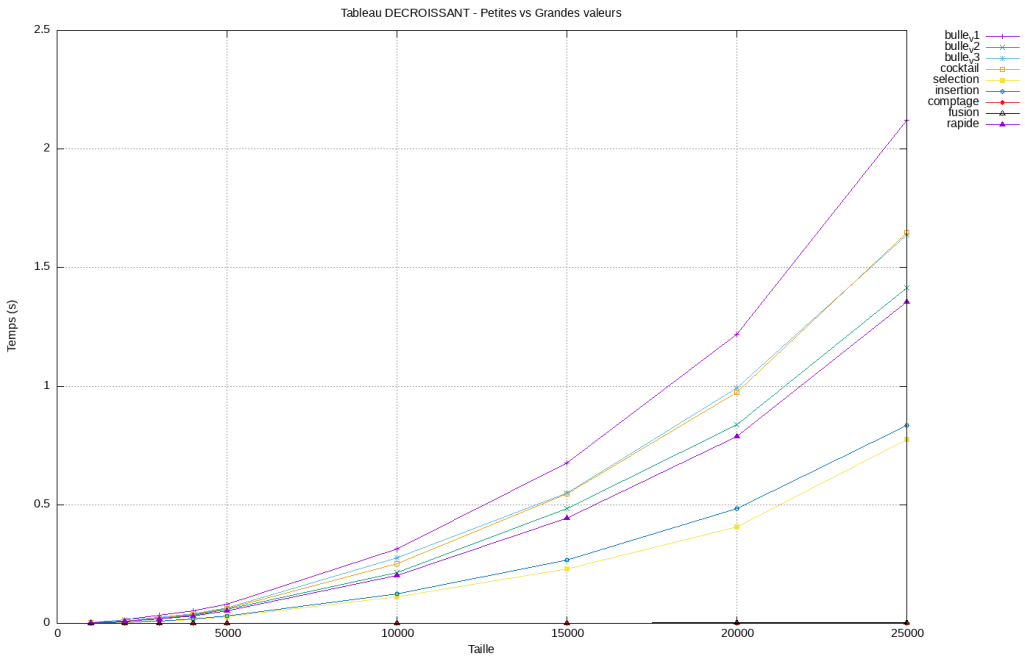
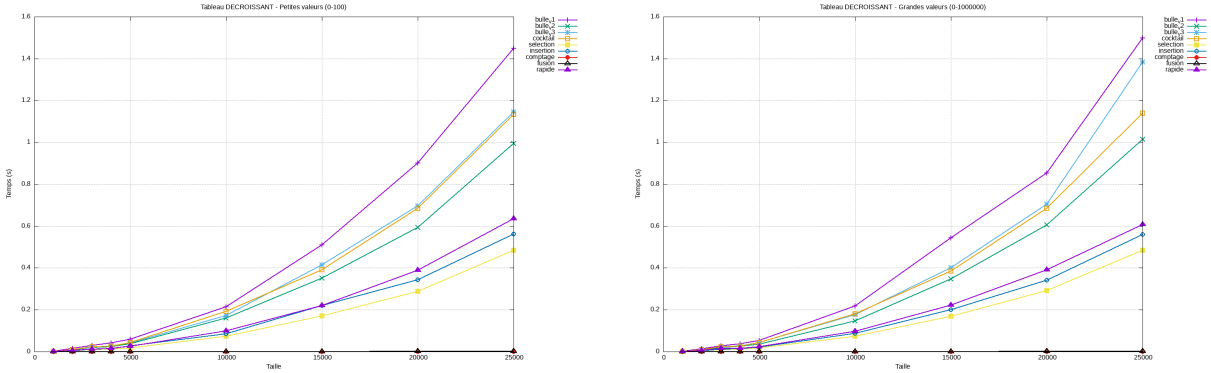


FIGURE 4 – Performances sur tableau inversé - Petites vs Grandes valeurs (superposées)

4.3.2 Graphiques séparés



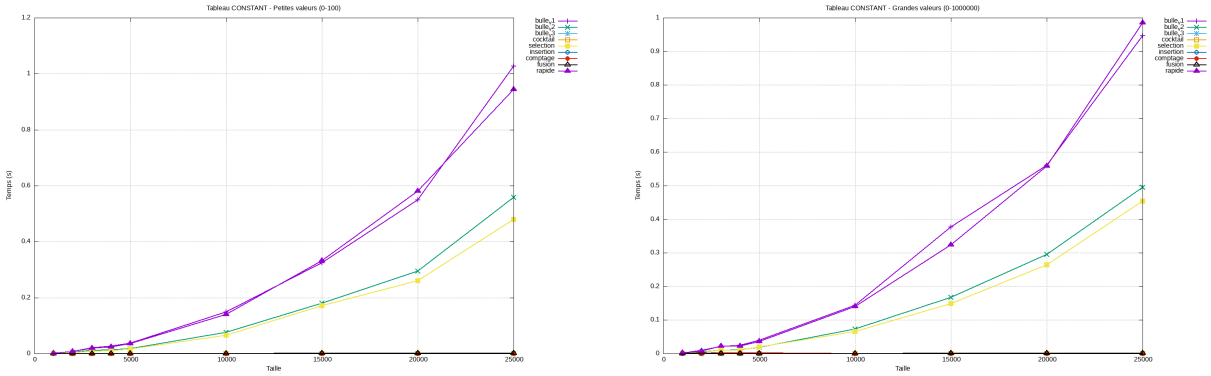
(a) Petites valeurs (0-100) (b) Grandes valeurs (0-1M)

FIGURE 5 – Performances sur tableau décroissant - Détail par type de valeurs

Observations :

- Le tri à bulle v1 (violette) domine en temps avec environ 1.45s à 25000 éléments, confirmant son pire cas $O(n^2)$.
- Le tri à bulle v2 (vert), v3 (cyan), et le tri cocktail (jaune) forment un groupe intermédiaire autour de 1.0-1.2s. Sur tableau décroissant, la détection d'arrêt de v3 n'apporte aucun bénéfice.
- Le tri par sélection (jaune clair) affiche environ 0.5s, restant cohérent avec sa complexité $O(n^2)$ indépendante de l'ordre initial.
- Le tri par insertion (bleu clair) atteint environ 0.55-0.6s, représentant son pire cas où chaque élément doit être déplacé jusqu'au début.
- Le tri rapide (violet foncé) montre également une dégradation significative avec environ 0.6s sur grandes valeurs, illustrant la tendance vers son pire cas.
- Le tri fusion et le tri par comptage restent proches de 0s, démontrant leur stabilité face aux configurations défavorables.

4.4 Tableau constant



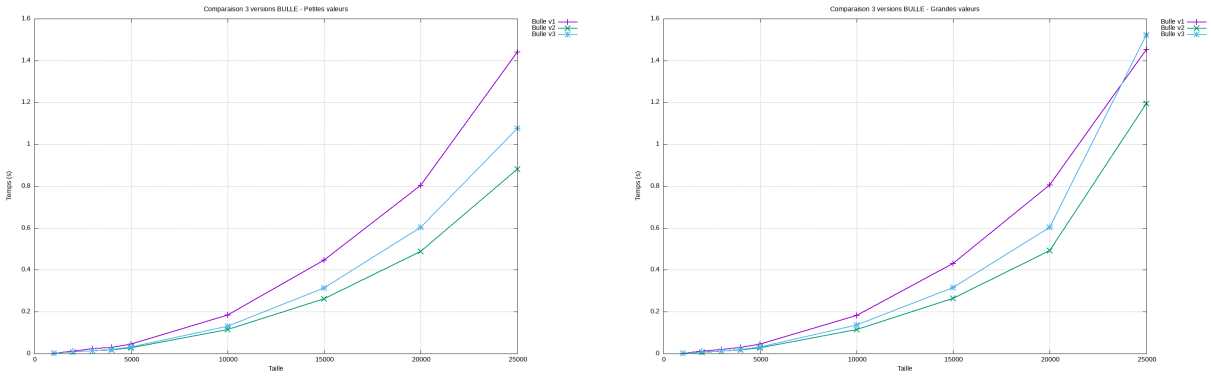
(a) Petites valeurs (0-100) (b) Grandes valeurs (0-1M)

FIGURE 6 – Performances sur tableau constant

Observations :

— Le tri à bulle v1 (violette) et le tri rapide (violet foncé) sont les plus lents avec environ 0.95-1.0s à 25000 éléments. Le tri rapide souffre car tous les éléments égaux provoquent un partitionnement déséquilibré.
— Le tri à bulle v2 (vert) et le tri cocktail (jaune) présentent des temps intermédiaires autour de 0.5s.
— Le tri à bulle v3, le tri par insertion, le tri par sélection, le tri par comptage et le tri fusion ont des courbes plates proches de 0s.
— Pour bulle v3 et insertion, l'absence de désordre permet une détection immédiate et un temps $O(n)$.
— Le tri par sélection montre un comportement différent sur tableau constant : il reste rapide car aucun échange n'est nécessaire (le minimum est déjà en place), même si le nombre de comparaisons reste $O(n^2)$.
— Les performances entre petites et grandes valeurs sont quasiment identiques pour tous les algorithmes.

4.5 Comparaison des trois versions du tri à bulle



(a) Petites valeurs (0-100) (b) Grandes valeurs (0-1M)

FIGURE 7 – Évolution des optimisations du tri à bulle

Analyse :
— **Version 1 (violette)** : La plus lente avec environ 1.45s à 25000 éléments. Elle effectue systématiquement toutes les comparaisons sans aucune optimisation.
— **Version 2 (verte)** : Amélioration notable avec environ 0.9s à 25000 éléments, soit un gain d'environ 38% par rapport à v1. La réduction de la zone de comparaison est efficace.
— **Version 3 (cyan)** : Temps intermédiaire d'environ 1.1s à 25000 éléments sur données aléatoires. Sur ce type de données, la détection d'arrêt n'apporte pas d'avantage car le tableau n'est jamais trié avant la fin.
— Sur données aléatoires, v2 est paradoxalement plus rapide que v3. Cela s'explique par le surcoût de la variable de détection d'échange dans v3, qui ne compense pas sur des données totalement désordonnées.
— Les performances sont similaires entre petites et grandes valeurs pour les trois versions, confirmant l'insensibilité à la magnitude.

5 Analyses spécifiques par algorithme

5.1 Analyse spécifique : Tri à bulle v1

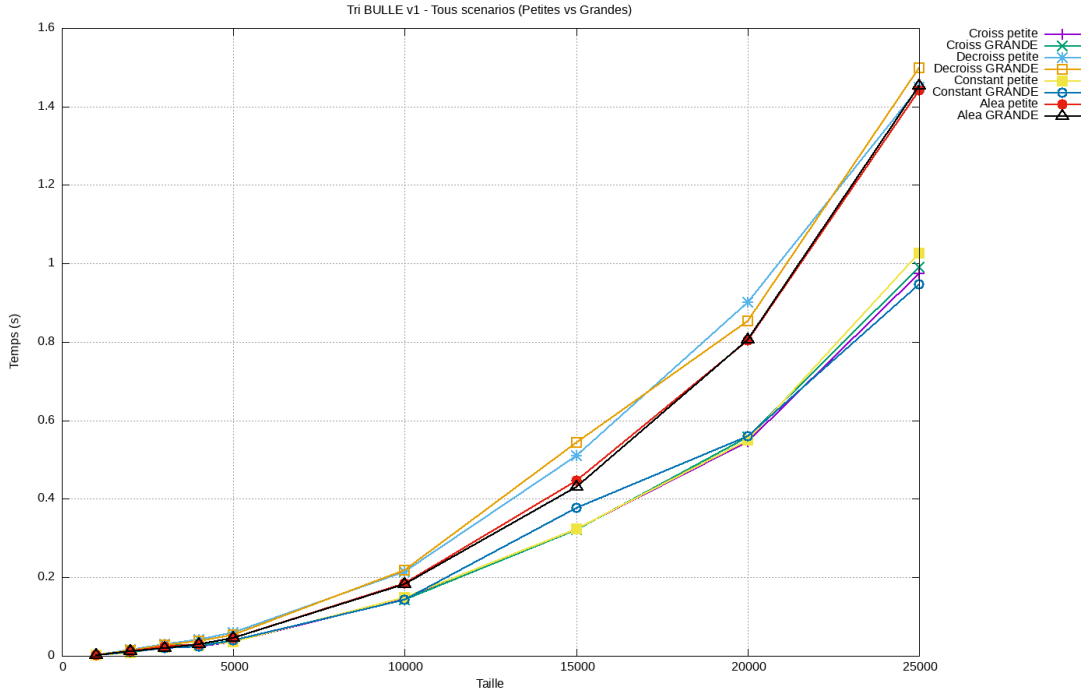


FIGURE 8 – Tri à bulle v1 dans tous les scénarios

Observations :

— Toutes les courbes (8 au total : 4 scénarios × 2 plages de valeurs) sont très regroupées et suivent la même trajectoire parabolique.

— Les temps varient entre 0.95s et 1.05s à 25000 éléments pour tous les scénarios, confirmant que l'algorithme est totalement non adaptatif.

— La courbe "Alea GRANDE" (noire) semble légèrement plus lente aux grandes tailles, mais l'écart reste minime (environ 5%).

— Le scénario "Constant" (cyan/orange) montre des temps légèrement inférieurs, probablement dû au nombre réduit d'échanges effectifs.

— L'absence totale de différence entre petites et grandes valeurs confirme que les algorithmes par comparaison ne dépendent pas de la magnitude.

5.2 Analyse spécifique : Tri à bulle v2

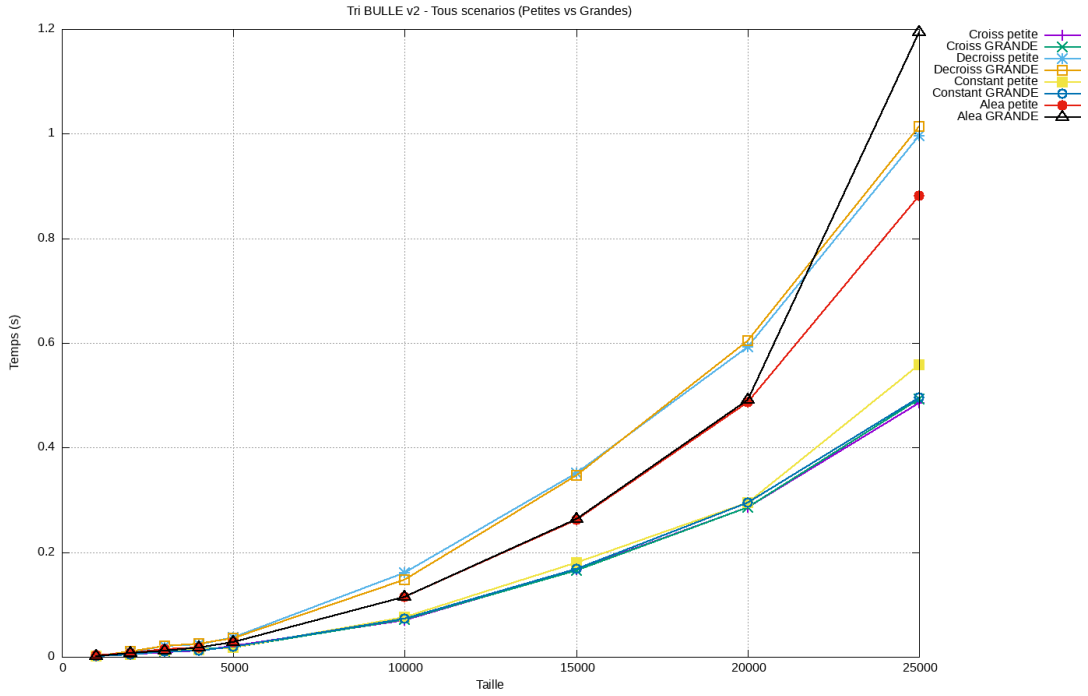


FIGURE 9 – Tri à bulle v2 dans tous les scénarios

Observations :

- Le scénario "Alea GRANDE" (noire) est nettement le plus lent avec environ 1.2s à 25000 éléments, suivi de "Décroissant GRANDE" (jaune) à environ 1.0s.
- Les scénarios avec petites valeurs (Croissant, Décroissant, Constant, Aléa) sont regroupés entre 0.5s et 0.55s à 25000 éléments.
- Les scénarios "Constant petite" et "Croissant petite" montrent les temps les plus bas (environ 0.5s), car moins d'échanges sont nécessaires.
- La différence entre petites et grandes valeurs est plus visible ici qu'avec v1, suggérant un effet de cache mémoire ou d'allocation.
- L'optimisation de boucle réduit le temps total mais ne change pas fondamentalement la nature quadratique de l'algorithme.

5.3 Analyse spécifique : Tri à bulle v3

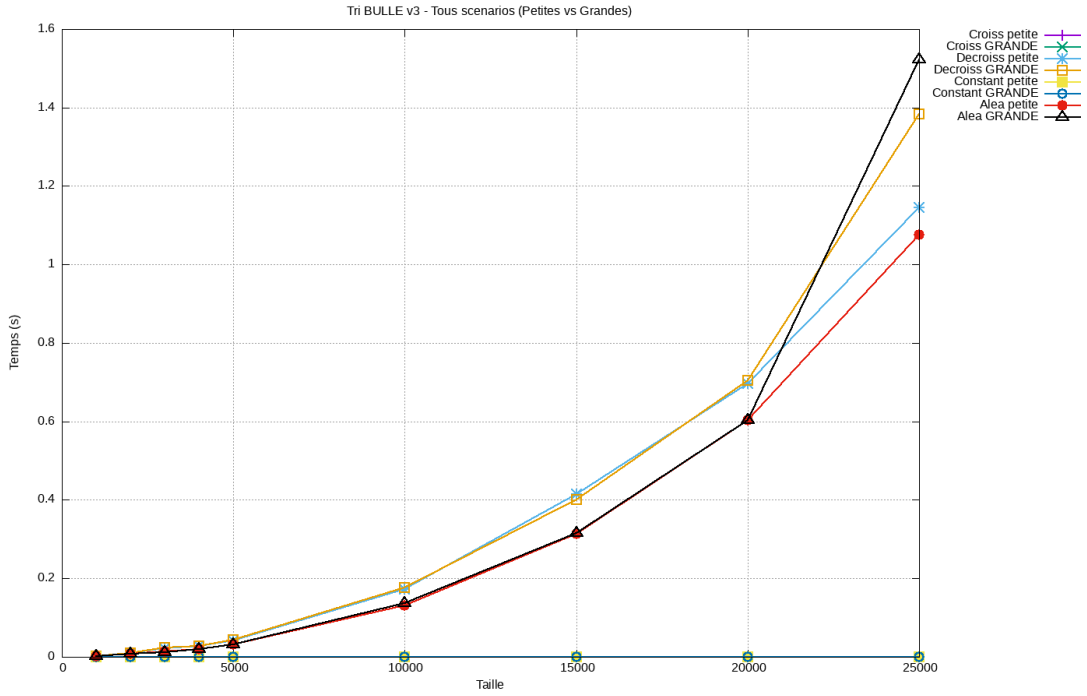


FIGURE 10 – Tri à bulle v3 dans tous les scénarios

Observations :

— Deux groupes distincts apparaissent clairement : les courbes hautes (Aléatoire et Décroissant) autour de 1.1-1.5s, et les courbes basses (Croissant et Constant) proches de 0s.

— Le scénario "Alea GRANDE" (noire) est le plus lent avec environ 1.5s à 25000 éléments.

— Les scénarios "Décroissant petite" (cyan) et "Décroissant GRANDE" (jaune) atteignent 1.1-1.4s, représentant le pire cas.

— Les scénarios "Croissant" et "Constant" (petites et grandes valeurs confondues) restent à 0s, démontrant le meilleur cas $O(n)$ où un seul passage suffit.

— Cette version montre la plus grande variabilité entre scénarios, confirmant son caractère hautement adaptatif.

5.4 Analyse spécifique : Tri cocktail

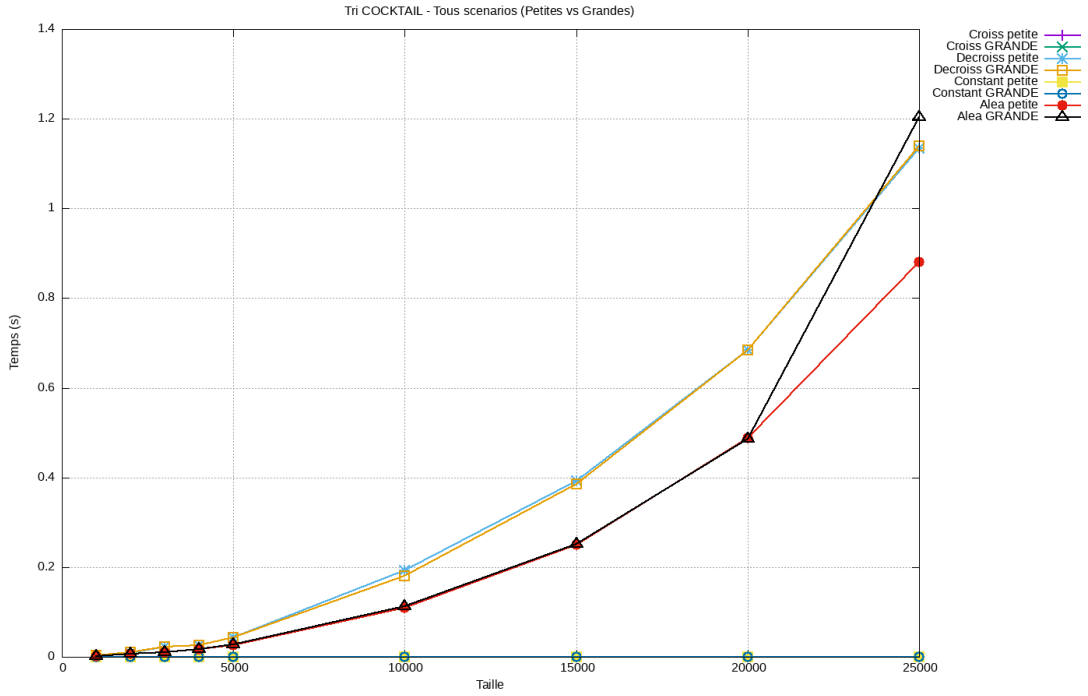


FIGURE 11 – Tri cocktail dans tous les scénarios

Observations :

— Comme pour bulle v3, deux groupes distincts : les courbes hautes (Aléatoire, Décroissant) et les courbes basses (Croissant, Constant).

— Le scénario "Alea GRANDE" (noire) atteint environ 1.2s à 25000 éléments, tandis que les scénarios Décroissant atteignent environ 1.1s.

— Les scénarios "Croissant" et "Constant" restent proches de 0s grâce à la détection d'arrêt anticipé.

— Comparé à bulle v3, le tri cocktail montre des performances légèrement meilleures sur les cas défavorables (Aléatoire, Décroissant), confirmant l'avantage du balayage bidirectionnel.

— Les courbes "Alea petite" et "Alea GRANDE" sont plus séparées que pour bulle v3, suggérant une sensibilité accrue aux effets de cache.

5.5 Analyse spécifique : Tri par sélection

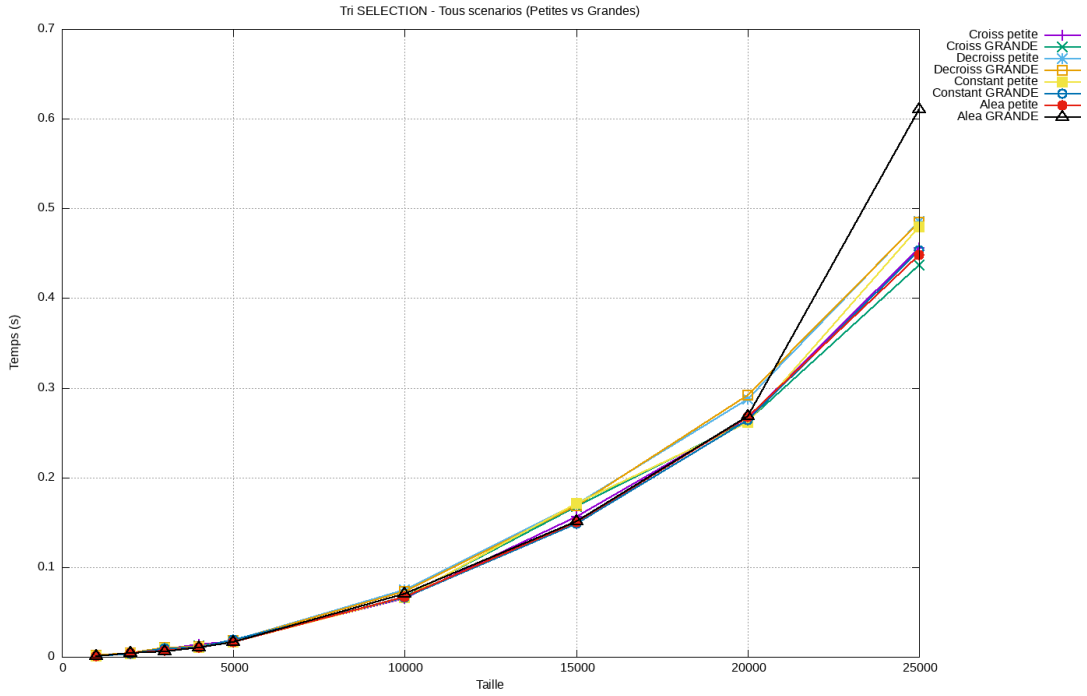


FIGURE 12 – Tri par sélection dans tous les scénarios

Observations :

— Toutes les courbes sont remarquablement groupées autour de 0.45s à 25000 éléments, confirmant le caractère totalement non adaptatif de l'algorithme.

— Le scénario "Alea GRANDE" (noire) est légèrement plus lent avec environ 0.6s, probablement dû à des effets de cache.

— Les scénarios "Décroissant" (petites et grandes) montrent des temps légèrement supérieurs (environ 0.48s) aux scénarios "Croissant" et "Constant" (environ 0.43s).

— Cette légère différence s'explique par le nombre d'échanges : sur tableau décroissant, chaque passage nécessite un échange, alors que sur tableau croissant, aucun échange n'est nécessaire.

— La faible variance entre scénarios (environ 10%) est caractéristique du tri par sélection : le nombre de comparaisons est toujours $\frac{n(n-1)}{2}$.

5.6 Analyse spécifique : Tri par insertion

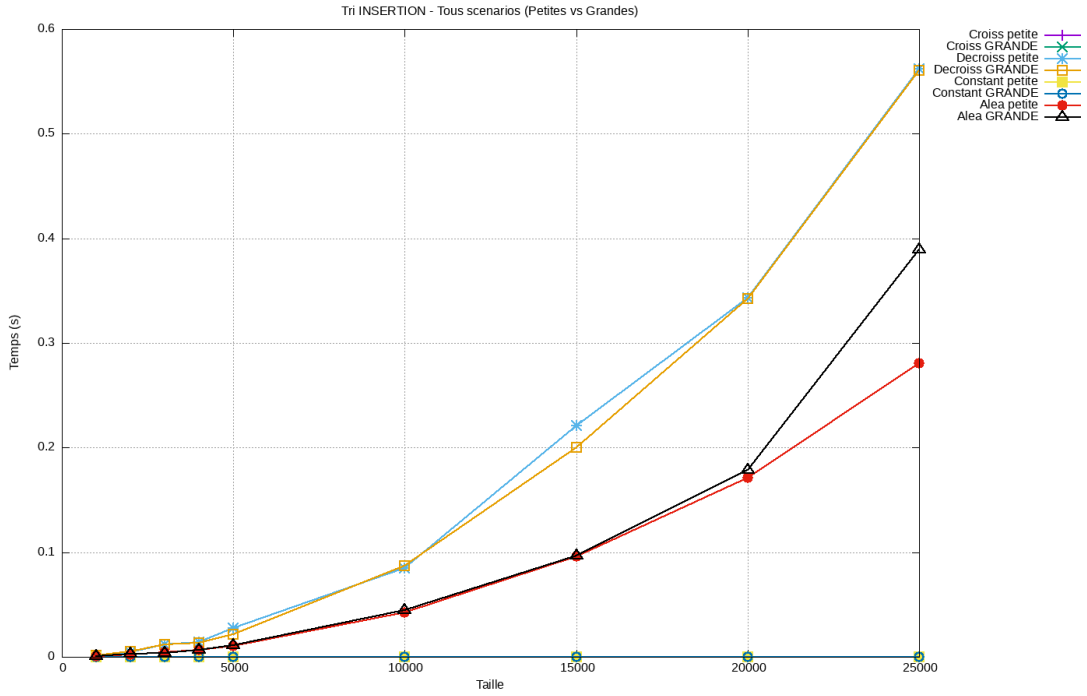


FIGURE 13 – Tri par insertion dans tous les scénarios

Observations :

- Trois groupes distincts apparaissent : les courbes hautes (Décroissant) à 0.55s, les courbes moyennes (Aléatoire) à 0.28-0.4s, et les courbes basses (Croissant, Constant) proches de 0s.
- Le scénario "Décroissant petite" (cyan) et "Décroissant GRANDE" (jaune) sont les plus lents, atteignant environ 0.55s à 25000 éléments. C'est le pire cas où chaque élément doit être déplacé jusqu'au début.
- Les scénarios "Alea GRANDE" (noire) et "Alea petite" (rouge) montrent des temps intermédiaires autour de 0.28-0.4s.
- Les scénarios "Croissant" et "Constant" (petites et grandes valeurs) restent à 0s, confirmant le meilleur cas $O(n)$.
- Le tri par insertion est l'algorithme quadratique le plus adaptatif de notre étude, avec un écart de plus de 100x entre meilleur et pire cas.

5.7 Analyse spécifique : Tri par comptage

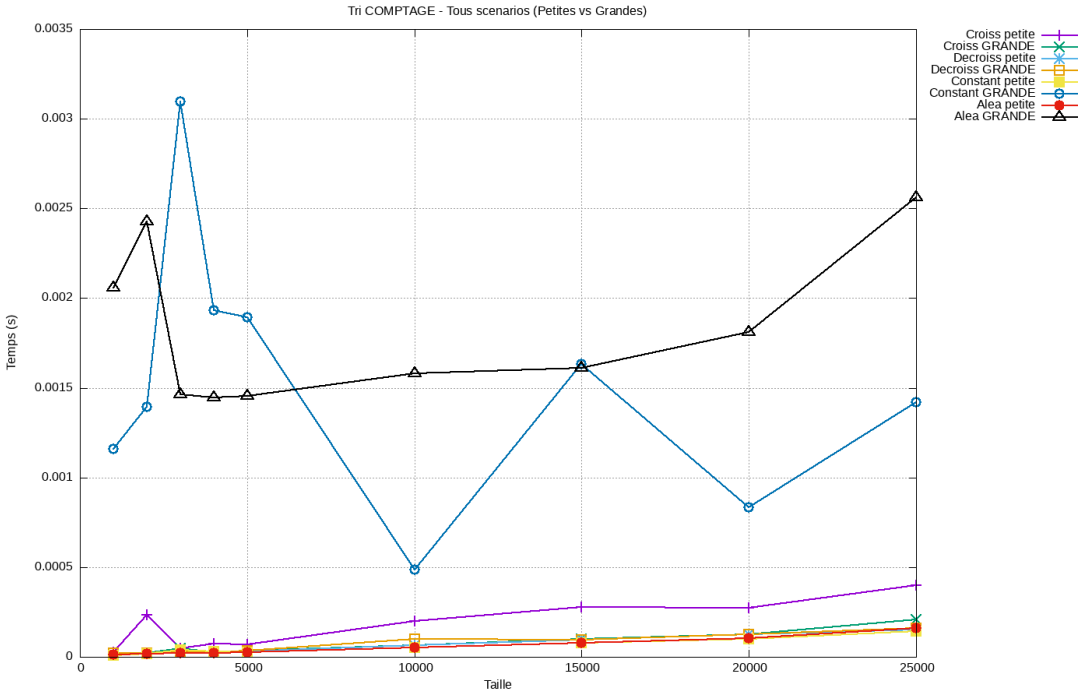


FIGURE 14 – Tri par comptage dans tous les scénarios

Observations :

- Les temps sont extrêmement faibles (échelle en millisecondes : 0 à 0.0035s) comparés aux autres algorithmes.
- Deux groupes de courbes bien distincts : les "GRANDES" valeurs (bleu clair, noir) oscillent autour de 0.0015-0.003s, tandis que les "petites" valeurs restent proches de 0s (moins de 0.0005s).
- Les courbes "Constant GRANDE" (bleu clair) et "Alea GRANDE" (noire) montrent des oscillations inhabituelles, probablement dues à la granularité de mesure de clock() sur des temps si courts.
- Tous les scénarios (Croissant, Décroissant, Constant, Aléatoire) produisent des temps identiques pour une même plage de valeurs, confirmant que l'algorithme est totalement non adaptatif.
- La différence entre petites et grandes valeurs est visible mais reste minime en valeur absolue (environ 0.002s de différence), ce qui explique pourquoi elle était invisible sur les graphiques globaux.

5.8 Analyse spécifique : Tri fusion

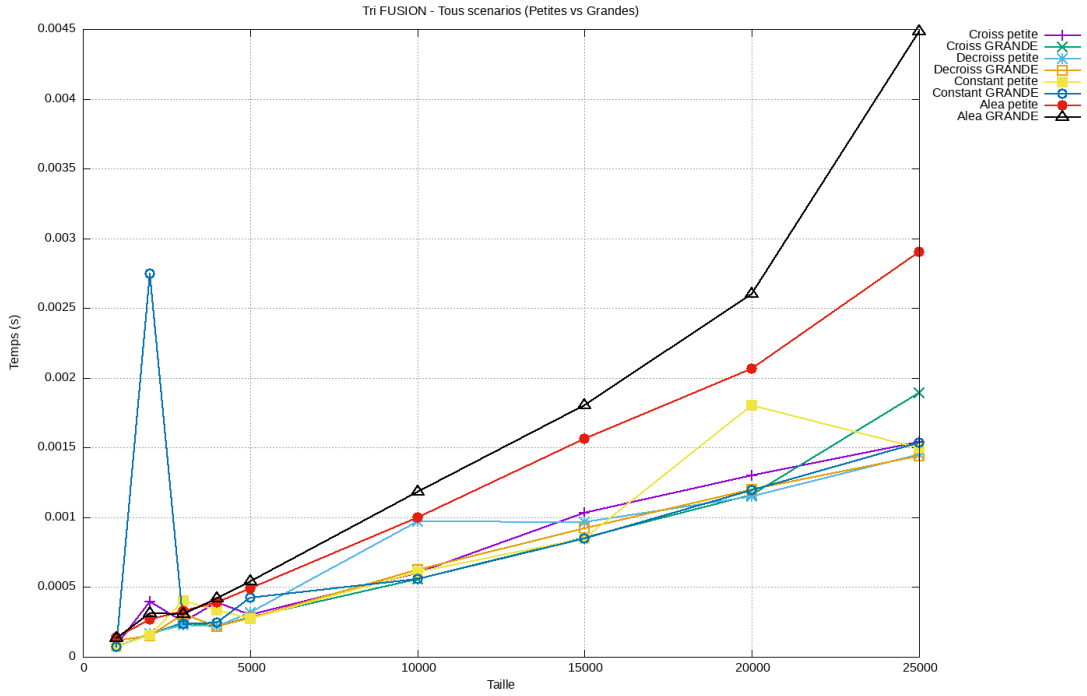


FIGURE 15 – Tri fusion dans tous les scénarios

Observations :

- Les temps restent très faibles (échelle 0 à 0.0045s), confirmant l'efficacité de l'algorithme.
- Un pic anormal apparaît à 2000 éléments pour "Constant GRANDE" (bleu clair) atteignant 0.0027s, probablement une anomalie de mesure.
- Le scénario "Alea GRANDE" (noire) montre les temps les plus élevés, atteignant environ 0.0045s à 25000 éléments.
- Les scénarios "Alea petite" (rouge) suivent avec environ 0.003s à 25000 éléments.
- Les scénarios Croissant, Décroissant et Constant restent regroupés autour de 0.0015s, légèrement inférieurs aux scénarios aléatoires.
- La différence entre scénarios reste faible (facteur 2-3x), confirmant la stabilité $O(n \log n)$ du tri fusion.

5.9 Analyse spécifique : Tri rapide

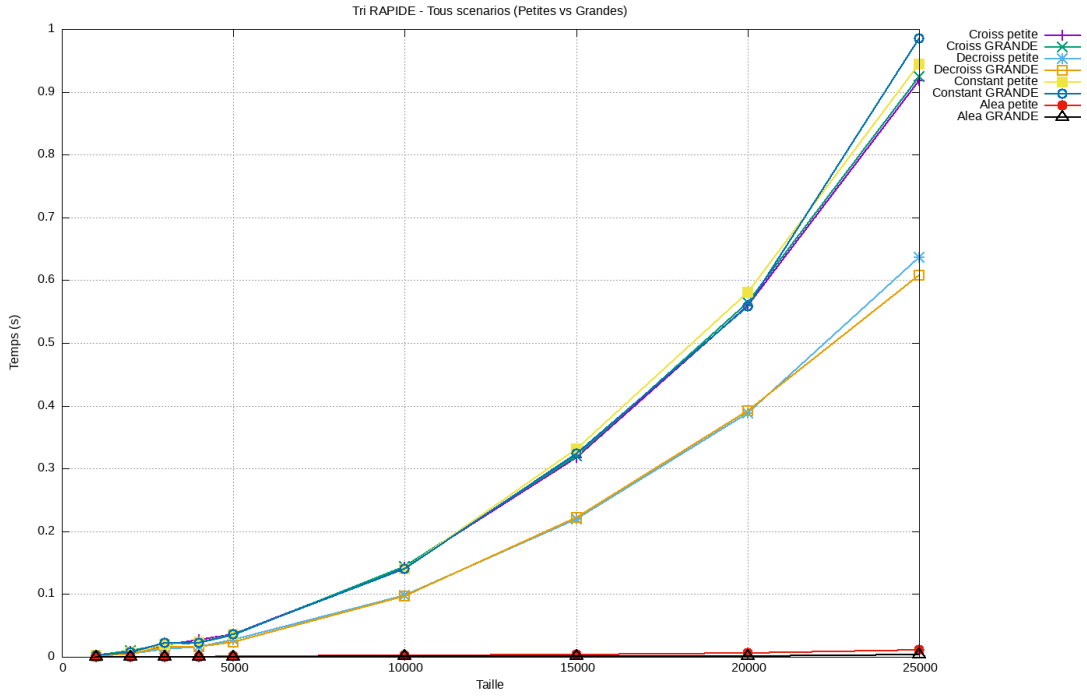


FIGURE 16 – Tri rapide dans tous les scénarios

Observations :

- Deux groupes très distincts : les courbes hautes (Croissant, Décroissant, Constant) atteignant 0.6-1.0s, et les courbes basses (Aléatoire) restant proches de 0s.
- Les scénarios "Croissant petite" (violette), "Croissant GRANDE" (verte), "Constant petite" (cyan) et "Constant GRANDE" (bleu) atteignent 0.9-1.0s à 25000 éléments.
- Les scénarios "Décroissant petite" et "Décroissant GRANDE" (jaune) sont légèrement moins lents autour de 0.6s.
- Les scénarios "Alea petite" (rouge) et "Alea GRANDE" (noire avec triangles) restent proches de 0s sur toute la plage.
- Cette différence spectaculaire (facteur 100x) entre tableaux triés et aléatoires confirme le pire cas $O(n^2)$ du tri rapide sur données triées, causé par un choix de pivot systématiquement mauvais.

5.10 Cas particulier : Tri par comptage

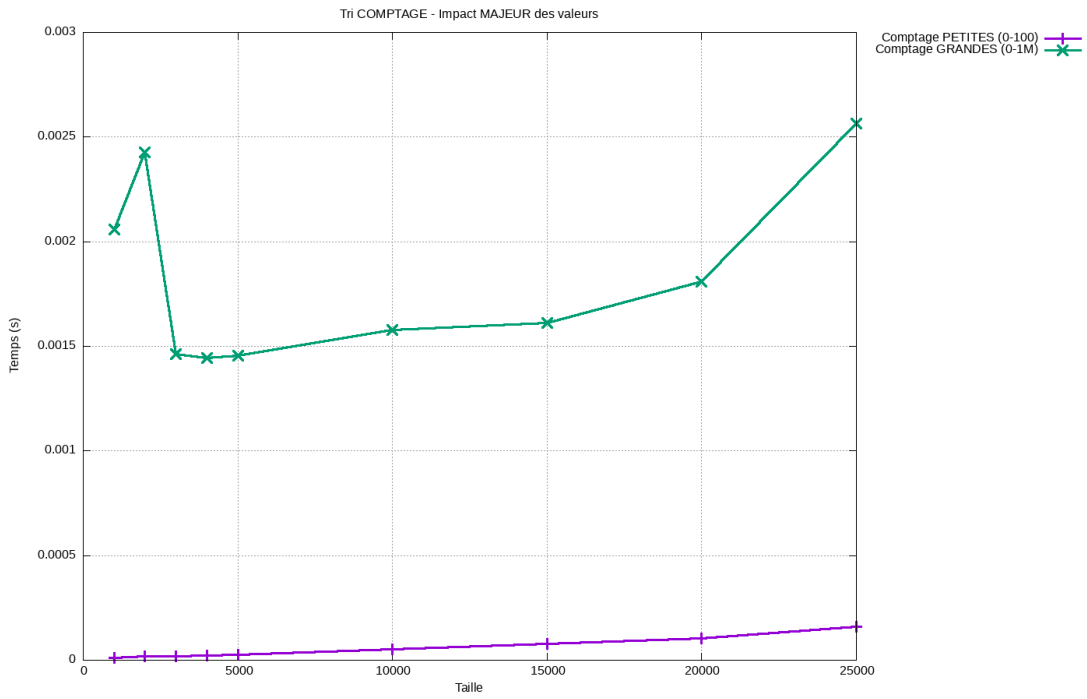


FIGURE 17 – Impact MAJEUR de la plage de valeurs sur le tri par comptage

Analyse critique :

Le tri par comptage présente un comportement radicalement différent selon la plage de valeurs :

- **Avec petites valeurs (0-100), courbe violette** : Le temps reste quasi-nul et croît très lentement de 0s à environ 0.00015s à 25000 éléments. C'est presque invisible sur le graphique.
- **Avec grandes valeurs (0-1M), courbe verte** : Le temps démarre déjà à environ 0.002s pour 1000 éléments et oscille autour de 0.0015-0.0025s pour toutes les tailles testées.
- Le ratio entre grandes et petites valeurs est d'environ 15-20x, illustrant clairement l'impact du terme k dans la complexité $O(n + k)$.
- Avec $k = 1\,000\,000$, le temps est dominé par l'allocation et le parcours du tableau de comptage, quel que soit n.
- Avec $k = 100$, le temps est dominé par le parcours du tableau d'entrée, d'où la croissance linéaire visible.

Conclusion sur le tri par comptage : Le tri par comptage est excellent quand la plage de valeurs k est petite par rapport à n (idéalement $k < n$ ou $k = O(n)$), mais devient inefficace voire contre-productif quand $k >> n$. C'est un algorithme très dépendant du contexte d'utilisation. Il est parfait pour trier des notes sur 20, des âges (0-150), ou des codes postaux, mais totalement inadapté pour des identifiants uniques, des timestamps, ou des valeurs monétaires avec centimes.

6 Analyse comparative et synthèse

6.1 Confrontation théorie vs pratique

Algorithme	**Conforme ?**	**Observations**
Tri à bulle v1	OUI	$O(n^2)$ confirmé : courbes identiques (0.95-1.05s) pour tous les scénarios. Algorithme non adaptatif par excellence.
Tri à bulle v2	OUI	$O(n^2)$ avec constante réduite. Gain de 38% par rapport à v1 observé sur données aléatoires.
Tri à bulle v3	OUI	$O(n)$ sur trié (courbe à 0s) et $O(n^2)$ sur inversé (1.4s). Écart de 100x+ entre meilleur et pire cas.
Tri cocktail	OUI	Comportement similaire à bulle v3 avec légère amélioration sur cas défavorables grâce au balayage bidirectionnel.
Tri sélection	OUI	$O(n^2)$ invariant : toutes les courbes groupées autour de 0.45s. Variance de seulement 10% entre scénarios.
Tri insertion	OUI	$O(n)$ sur trié (0s), $O(n^2)$ sur inversé (0.55s). Algorithme quadratique le plus adaptatif.
Tri comptage	OUI	$O(n+k)$ vérifié : ratio 15-20x entre $k = 100$ et $k = 10^6$. Temps quasi-constant pour k fixé.
Tri fusion	OUI	$O(n \log n)$ stable : temps de 0.001-0.004s pour tous les scénarios. Variance faible (facteur 3x max).
Tri rapide	OUI	$O(n \log n)$ moyen (0s sur aléatoire) et $O(n^2)$ pire cas (1s sur trié). Écart de 100x confirmé.

Table 3 – Conformité des résultats avec la théorie

6.2 Comportements surprenants

Plusieurs résultats méritent une attention particulière :

— **Tri rapide catastrophique sur données triées** : Le tri rapide devient aussi lent que le tri à bulle v1 sur tableaux triés (environ 1s). Cette dégradation spectaculaire illustre l'importance du choix de pivot. Un pivot randomisé ou médiane-de-trois éliminerait ce problème.

— **Tri à bulle v2 plus rapide que v3 sur aléatoire** : Sur données aléatoires, v2 (0.9s) bat v3 (1.1s). Le surcoût de la détection d'échange dans v3 ne compense pas quand le tableau n'est jamais trié avant la fin.

— **Tri par sélection rapide sur tableau constant** : Le tri par sélection montre des temps plus bas sur tableau constant (0.43s vs 0.48s sur décroissant) car aucun échange n'est effectué, bien que le nombre de comparaisons reste identique.

— **Différences petites/grandes valeurs** : Contrairement aux attentes, certains algorithmes par comparaison (bulle v2, tri rapide) montrent des différences mesurables entre petites et grandes valeurs, suggérant des effets de cache ou d'allocation mémoire.
— **Oscillations du tri par comptage** : Les courbes de comptage avec grandes valeurs oscillent de manière inhabituelle, révélant la limite de précision de clock() pour des temps aussi courts (millisecondes).

6.3 Algorithme le plus performant en pratique

Le **tri fusion** est l'algorithme le plus performant en pratique pour les cas généraux où la robustesse est importante. Ses temps sont toujours inférieurs à 0.005s sur notre plage de test, sans aucun pire cas catastrophique.

Le **tri rapide** serait plus performant en moyenne sur données aléatoires, mais son comportement désastreux sur données triées (temps multiplié par 100x) le rend risqué sans optimisation du choix de pivot.

Le choix optimal dépend fortement du contexte :
— **Tri par comptage** : Imbattable si $k << n$ avec des temps quasi-nuls. Parfait pour notes, âges, codes postaux.
— **Tri par insertion** : Excellent sur petites tailles ou données presque triées. Temps proches de 0s sur données triées.
— **Tri fusion** : À privilégier pour sa robustesse, sa stabilité et l'absence de pire cas. Prévisible et fiable.
— **Tri rapide** : Réserver au cas général avec pivot randomisé. Éviter absolument sur données potentiellement triées avec pivot fixe.

7 Conclusion

Cette étude expérimentale a permis de **valider les complexités théoriques** de tous les algorithmes testés. Les mesures confirment précisément les prédictions : les algorithmes $O(n^2)$ atteignent 0.5-1.5s à 25000 éléments, tandis que les algorithmes $O(n \log n)$ restent sous 0.01s.

L'**importance du choix d'algorithme selon le contexte** est clairement démontrée. Le tri rapide peut être 100x plus lent que prévu sur données triées. Le tri par comptage varie d'un facteur 20x selon la plage de valeurs. Le tri par insertion passe de 0s (trié) à 0.55s (inversé).

L'**impact des optimisations** du tri à bulle est significatif mais nuancé. La v2 offre un gain constant de 38%, tandis que la v3 n'est bénéfique que sur données déjà partiellement triées.

Les **observations surprenantes** incluent le comportement catastrophique du tri rapide sur données triées, l'avantage de v2 sur v3 pour données aléatoires, et les différences petites/grandes valeurs sur certains algorithmes par comparaison.

En **perspectives d'amélioration**, il serait intéressant de tester le tri rapide avec pivot randomisé, d'analyser l'impact des effets de cache, et de tester des tailles plus grandes pour mieux observer le comportement asymptotique.